Classe `MultiBootMixin`

A classe `MultiBootMixin` está em `maasserver/api/multiboot_mixin.py`.

```
import logging

from django.core.exceptions import PermissionDenied

from maasserver.api.support import operation
from maasserver.enum import NODE_STATUS, NODE_STATUS_CHOICES_DICT
from maasserver.exceptions import (
    MAASAPIBadRequest,
    MAASAPIValidationError,
    NodeStateViolation,
)
from maasserver.permissions import NodePermission
from maasserver.forms import MachineForm
from maasserver.models.node import Node
from maasserver.models.nodemetadata import NodeMetadata
from maasserver.node_status import NODE_TRANSITIONS
from maasserver import locks
from maasserver.multiboot_deploy import (
    compose_os_config,
    delete_layout,
    generate_multi_boot_installer,
    load_layout,
    read_layout_from_request,
    store_layout,
    validate_layout,
)

maaslog = logging.getLogger("maas")


class MultiBootMixin:
    """Mixin that adds multi-boot operations to MachineHandler."""

    @classmethod
    def multi_boot(handler, node) -> bool:
        """Return True if this node has a multi-boot layout configured."""
        return load_layout(node) is not None

    @operation(idempotent=False)
    def set_layout(self, request, system_id) -> Node:
        machine = self.model.objects.get_node_or_404(
            system_id=system_id, user=request.user,
            perm=NodePermission.edit,
        )
        if not request.user.has_perm(NodePermission.edit, machine):
            raise PermissionDenied()
        layout_data = read_layout_from_request(request)
        validate_layout(layout_data)
        store_layout(machine, layout_data)
        return machine

    @operation(idempotent=True)
    def get_layout(self, request, system_id) -> dict:
```

```
    machine = self.model.objects.get_node_or_404(
        system_id=system_id, user=request.user,
        perm=NodePermission.view,
    )
    if not request.user.has_perm(NodePermission.view, machine):
        raise PermissionDenied()
    layout = load_layout(machine)
    if layout is None:
        raise MAASAPIBadRequest(
            "No multi-boot layout configured for this machine."
        )
    return layout

@operation(idempotent=False)
def delete_layout(self, request, system_id) -> Node:
    machine = self.model.objects.get_node_or_404(
        system_id=system_id, user=request.user,
        perm=NodePermission.edit,
    )
    if not request.user.has_perm(NodePermission.edit, machine):
        raise PermissionDenied()
    delete_layout(machine)
    return machine

@operation(idempotent=False)
def multi_boot_deploy(self, request, system_id) -> Node:
    # Permission checks (mirrors deploy())
    machine = self.model.objects.get_node_or_404(
        system_id=system_id, user=request.user,
        perm=NodePermission.edit,
    )
    if not request.user.has_perm(NodePermission.edit, machine):
        raise PermissionDenied()

    # Acquire (mirrors deploy())
    if machine.status == NODE_STATUS.READY:
        with locks.node_acquire:
            if machine.owner is not None \
                    and machine.owner != request.user:
                raise NodeStateViolation(
                    "Can't allocate a machine belonging to another user."
                )
            machine.acquire(request.user)

    # State check (mirrors deploy())
    if NODE_STATUS.DEPLOYING \
            not in NODE_TRANSITIONS[machine.status]:
        raise NodeStateViolation(
            "Can't deploy a machine in the '%s' state"
            % NODE_STATUS_CHOICES_DICT[machine.status]
        )

    # Load and validate layout
    layout = load_layout(machine)
    if layout is None:
        raise MAASAPIBadRequest(
```

```
        "No multi-boot layout configured for this machine."
    )
validate_layout(layout)

# Generate per-OS curtin configs and embed in wrapper script
os_configs = []
for i, os_entry in enumerate(layout["oses"]):
    config = compose_os_config(
        request, machine, os_entry, i
    )
    os_configs.append(config)

script = generate_multi_boot_installer(
    layout, machine, os_configs
)
NodeMetadata.objects.update_or_create(
    node=machine, key="multi_boot_preseed",
    defaults={"value": script},
)

# Form init
form = MachineForm(instance=machine, data={})
if form.is_valid():
    form.save()
else:
    raise MAASAPIValidationError(form.errors)

# Power on
return self.power_on(request, system_id)
```